

IT112 Web System and Technologies
Phase 2 - Backend Development
Lab Report by SURVIVORS

Group Members:

Jexson Sedon
John Roan Ballester
Michelle Diaz
Judy Pearl Balictar
Alyssa Jenille Reantaso
Ronel Garcia

Submitted to:

Guillermo V. Red,Jr,DIT
Assistant Professor IV



I. Overview of Backend Setup

The Attendance System backend is built using Node.js, Express.js, and MongoDB. It follows a modular structure where routes handle user requests, controllers process the logic, models define the data structure, and the database stores the information. This setup makes the system organized, easy to manage, and capable of handling basic operations such as adding, viewing, updating, and deleting attendance records.

II. Code Snippets

A. server.js

```
const dns = require('dns');
// Force Cloudflare and Google DNS servers
dns.setServers(['1.1.1.1', '8.8.8.8']);

// Then your existing mongoose connection code
const mongoose = require('mongoose');
// ... rest of your code

require('dotenv').config();
const express = require('express');
const cors = require('cors');
const connectDB = require('./config/db');

const app = express();

// Connect DB
connectDB();

// Middleware
app.use(cors());
app.use(express.json());

// Routes
app.use('/api/auth', require('./routes/authRoutes'));
app.use('/api/attendance', require('./routes/attendanceRoutes'));

const PORT = process.env.PORT || 3000;

app.listen(PORT, () => {
    console.log(`Server running on port ${PORT}`);
});
```

This starts the server using Express.js and connects all parts of the system. It links the routes, connects to the database, and listens for requests.



B. Sample Model- Attendance.js

```
const mongoose = require('mongoose');

const attendanceSchema = new mongoose.Schema({
  studentId: { type: String, required: true },
  date: { type: Date, default: Date.now },
  status: {
    type: String,
    enum: ['Early', 'On-Time', 'Late', 'Absent']
  },
  timeIn: { type: Date }
}, { timestamps: true });

module.exports = mongoose.model('Attendance', attendanceSchema);
```

The model defines the structure of the data using Mongoose. It ensures that attendance records are organized and consistent in the database.

C. Sample Routes- attendance.Routes.js

```
const router = require('express').Router();
const {
  checkIn,
  getAll,
  update,
  delete: deleteAttendance
} = require('../controllers/attendanceController');

const { protect, authorize } = require('../middleware/authMiddleware');

// Student can check-in
router.post('/checkin', protect, authorize('student'), checkIn);

// Only teacher/admin can view all
router.get('/', protect, authorize('teacher', 'admin'), getAll);

// Admin only update/delete
router.put('/:id', protect, authorize('admin'), update);
router.delete('/:id', protect, authorize('admin'), deleteAttendance);

module.exports = router;
```

Routes handle incoming requests and send them to the correct functions. They define actions like adding, viewing, updating, and deleting attendance.



D. Sample Controller- attendance.Controller.js

```
const Attendance = require('../models/Attendance');

// Auto classification
const classifyTime = (time) => {
  const hour = new Date(time).getHours();

  if (hour < 8) return "Early";
  if (hour === 8) return "On-Time";
  if (hour > 8 && hour < 10) return "Late";
  return "Absent";
};

// CREATE (Check-in)
exports.checkIn = async (req, res) => {
  const { studentId } = req.body;

  const now = new Date();
  const status = classifyTime(now);

  const record = new Attendance({
    studentId,
    timeIn: now,
    status
  });

  await record.save();
  res.json(record);
};

// READ
exports.getAll = async (req, res) => {
  const data = await Attendance.find();
  res.json(data);
};

// UPDATE
exports.update = async (req, res) => {
  const updated = await Attendance.findByIdAndUpdate(
    req.params.id,
    req.body,
    { new: true }
  );
  res.json(updated);
};
```



```
// DELETE
exports.delete = async (req, res) => {
  await Attendance.findByIdAndDelete(req.params.id);
  res.json({ message: "Deleted" });
};
```

The controller contains the main logic of the system. It processes requests, manages data, and performs actions like check-in and time classification.

III. API Testing

The API was tested to confirm successful data handling and system responses.

A. User – Register

The screenshot displays an API client interface with the following details:

- Method:** POST
- URL:** http://localhost:3000/api/auth/register
- Body (JSON):**

```
{
  "username": "John",
  "email": "john@gmail.com",
  "password": "john12345",
  "role": "student",
  "studentId": "STU12345"
}
```
- Status:** 200 OK
- Size:** 24 Bytes
- Time:** 195 ms
- Response (JSON):**

```
{
  "message": "Registered"
}
```
- Terminal:** Shows the command `npm run dev` being executed in a PowerShell window, with output indicating MongoDB is connected and the backend is running.



B. User – Login

The screenshot shows a REST client interface with the following details:

- Request:**
 - Method: POST
 - URL: `http://localhost:3000/api/auth/login`
 - Body (JSON):

```
1 {
2   "username": "John",
3   "email": "john@gmail.com",
4   "password": "john12345",
5   "role": "student",
6   "studentId": "STU12345"
7 }
```
- Response:**
 - Status: 200 OK
 - Size: 85 Bytes
 - Time: 173 ms
 - Body (JSON):

```
1 {
2   "message": "Login success",
3   "user": {
4     "id": "69e23a7cbfe78d747fdd53dd",
5     "role": "student"
6   }
7 }
```
- Terminal:** Shows the command `npm run dev` being executed in a terminal window.

C. Attendance – Check In

The screenshot shows a REST client interface with the following details:

- Request:**
 - Method: POST
 - URL: `http://localhost:3000/api/attendance/checkin`
 - Body (JSON):

```
1 {
2   "studentId": "STU2024003"
3 }
```
- Response:**
 - Status: 200 OK
 - Size: 233 Bytes
 - Time: 75 ms
 - Body (JSON):

```
1 {
2   "studentId": "STU2024003",
3   "status": "Absent",
4   "timeIn": "2026-04-17T14:15:15.281Z",
5   "_id": "69e24073bfe78d747fdd53e1",
6   "date": "2026-04-17T14:15:15.282Z",
7   "createdAt": "2026-04-17T14:15:15.282Z",
8   "updatedAt": "2026-04-17T14:15:15.282Z",
9   "v": 0
10 }
```



D. Attendance – Get All

GET ⌵ http://localhost:3000/api/attendance Send

Query Headers ⁴ Auth Body Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

1

Status: 200 OK Size: 937 Bytes Time: 53 ms Response ⌵

```
1  [
2    {
3      "_id": "69e24073bfe78d747fdd53e1",
4      "studentId": "STU2024003",
5      "status": "Absent",
6      "timeIn": "2026-04-17T14:15:15.281Z",
7      "date": "2026-04-17T14:15:15.282Z",
8      "createdAt": "2026-04-17T14:15:15.282Z",
9      "updatedAt": "2026-04-17T14:15:15.282Z",
10     "_v": 0
11   },
12   {
13     "_id": "69e2420ebfe78d747fdd53e2",
14     "studentId": "STU2024002",
15     "status": "Present",
16     "timeIn": "2026-04-17T14:15:15.281Z",
17     "date": "2026-04-17T14:15:15.282Z",
18     "createdAt": "2026-04-17T14:15:15.282Z",
19     "updatedAt": "2026-04-17T14:15:15.282Z",
20     "_v": 0
21   }
22 ]
```

Copy Response Chart

E. Attendance – Update

PUT ⌵ http://localhost:3000/api/attendance/69e24073bfe78d747fdd53e1 Send

Query Headers ⁴ Auth Body¹ Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

```
1  {
2    "status": "Excused",
3    "remarks": "Student was absent due to medical appointment"
4  }
```

Status: 200 OK Size: 234 Bytes Time: 79 ms Response ⌵

```
1  {
2    "_id": "69e24073bfe78d747fdd53e1",
3    "studentId": "STU2024003",
4    "status": "Excused",
5    "timeIn": "2026-04-17T14:15:15.281Z",
6    "date": "2026-04-17T14:15:15.282Z",
7    "createdAt": "2026-04-17T14:15:15.282Z",
8    "updatedAt": "2026-04-17T14:29:52.951Z",
9    "_v": 1
10 }
```

Copy Response Chart



F. Attendance – Delete

DELETE Send

Query Headers ⁴ Auth **Body** Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

```
1
```

Status: 200 OK Size: 21 Bytes Time: 88 ms Response

```
1 {
2   "message": "Deleted"
3 }
```

Response Chart

IV. Backed Folder Structure

Files

main

- backend
 - config
 - db.js
 - controllers
 - attendanceController.js
 - authController.js
 - middleware
 - authMiddleware.js
 - models
 - Attendance.js
 - User.js
 - routes
 - attendanceRoutes.js
 - authRoutes.js

final-project--The-Survivors--BSIT2a / backend / Add file ...

NooblsOnline modified: backend/README.md 8afe770 · 4 days ago History

Name	Last commit message	Last commit date
..		
config	new file: backend/.env	4 days ago
controllers	new file: backend/.env	4 days ago
middleware	new file: backend/.env	4 days ago
models	new file: backend/.env	4 days ago
routes	modified: backend/routes/attendanceRoutes.js	4 days ago
.env	new file: backend/.env	4 days ago
README.md	modified: backend/README.md	4 days ago
package-lock.json	modified: backend/package-lock.json	4 days ago
package.json	modified: backend/package-lock.json	4 days ago
server.js	new file: backend/.env	4 days ago



V. Group Member Contribution

MEMBERS	ROLE	CONTRIBUTIONS
Jexson Sedon	Project Manager	Even though the Project Manager was not directly involved in the technical tasks this week, I still observed that he provided guidance to ensure that our group stayed aligned with our overall system plan. He helped monitor our progress and made sure that the backend development followed the structure we planned in phase 1.
John Roan Ballester	Backend Manager	He focused on setting up the Express server and organizing the routes, controllers, and models. I documented how the routes were created and how they handled different requests in the system. When we encountered errors during integration, he helped fix backend issues so the system could function correctly.
Michelle Diaz	Database Manager	She worked on creating the Mongoose models and connecting the system to MongoDB. Based on my documentation, she ensured that the database connection was successful and that the data structure followed our planned design. I also captured how the data was stored after form submissions.
Judy Pearl Balictar	Frontend Manager	Although the Frontend Manager was not the main focus this week, she helped us review the frontend components to ensure they were ready for backend integration. She checked if the



		forms and inputs matched the required fields needed for submission to the server. This helped make the transition from frontend to backend smoother during testing and integration.
Alyssa Jenille Reantaso	Github Manager	She was responsible for organizing and maintaining our backend code in the repository. She also ensured a proper folder structure and clear commit messages for every update. This helped us keep track of changes and made our project more organized.
Ronel Garcia	Debugger/Tester and Documentation Officer	As the Documentation Officer, my role was to capture screenshots and clearly explain how the routes and system functionality worked. I documented each step of the process, including how data flows from the frontend to the backend and into the database. I also made sure that all outputs and features were properly recorded for our lab report.

